Supermarket Inventory Tracking System

Software Architecture Document

SWEN-755 Software Architecture

Rochester Institute of Technology

Team

Godson Umoren

Kobe Laprade

Leon Letournel

Mike Andrews

Revision 0.2 — First draft — 2026-10-18

Contents

1 Documentation Roadmap **1**
- 1.1 Document Management and Configuration Control Information . . . 1
- 1.2 Purpose and Scope of the SAD . . . 1
- 1.3 How the SAD Is Organized . . . 1
- 1.4 Stakeholder Representation . . . 2
- 1.5 Viewpoint Definitions . . . 2
  - 1.5.1 Components-and-Connectors Viewpoint Definition . . . 2
- 1.6 How a View Is Documented . . . 3
- 1.7 Relationship to Other SADs . . . 3
- 1.8 Process for Updating this SAD . . . 3

2 Architecture Background **4**
- 2.1 Problem Background . . . 4
  - 2.1.1 System Overview . . . 4
  - 2.1.2 Goals and Context . . . 4
  - 2.1.3 Significant Driving Requirements . . . 5
- 2.2 Solution Background . . . 10
  - 2.2.1 Architectural Approaches . . . 10
  - 2.2.2 Analysis Results . . . 13
  - 2.2.3 Requirements Coverage . . . 14
  - 2.2.4 Summary of Background Changes Reflected in Current Version . . . 15
- 2.3 Product Line Reuse Considerations . . . 15

3 Views **16**
- 3.1 Components-and-Connectors View . . . 16
  - 3.1.1 View description . . . 16
  - 3.1.2 View packet overview . . . 17
  - 3.1.3 Architecture background . . . 17
  - 3.1.4 Variability mechanisms . . . 17
  - 3.1.5 Primary presentation . . . 17
  - 3.1.6 Element catalog . . . 17
  - 3.1.7 Interfaces . . . 22
  - 3.1.8 Context diagram . . . 24
  - 3.1.9 Related view packets . . . 25
- 3.2 Module Decomposition View . . . 25
  - 3.2.1 View description . . . 25
  - 3.2.2 Primary presentation . . . 25
  - 3.2.3 Element catalog . . . 26
  - 3.2.4 Context diagram . . . 28
  - 3.2.5 Variability mechanisms . . . 28
  - 3.2.6 Architecture background . . . 28
  - 3.2.7 Related view packets . . . 28

4 Relations Among Views **28**

5 Referenced Materials **28**

6 Directory **29**
6.1 Index 29
6.2 Glossary 29
6.3 Acronym List 32

1 Documentation Roadmap

1.1 Document Management and Configuration Control Information

This section records the revision history of the document so that a reader can tell whether they are looking at the current version.

Revision	Date	Status	Summary of changes
0.1	2026-10-14	Skeleton	Initial compiled skeleton: every section present as a stub, boilerplate, or marked not applicable; shared element and operation names; the frozen scenario identifiers; a seeded glossary and references.
0.2	2026-10-18	First draft	First draft: C&C view, module decomposition view, IEP interface, requirements and utility tree, approaches and tactics.

1.2 Purpose and Scope of the SAD

This document specifies the software architecture of the Regional Supermarket Inventory Tracking System, which tracks inventory across a chain of supermarket stores and a central distribution center and drives the shipping orders that move product from the distribution center to the stores. It is the single place where the architecture is recorded. Following the SEI Views and Beyond template, we document the architecture as a set of views together with the information that applies across views. The draft utility tree (Table 3) rates every scenario and marks the two (H, H) scenarios whose delivery path this draft specifies.

We treat a design decision as architectural, and therefore record it here, when it concerns the elements of the system, their externally visible properties, and the relationships among them; we leave a decision to downstream design when it concerns only the private internals of a single element (Bass, Clements, and Kazman 2).

This first draft is a thin vertical slice. It carries two views. The module decomposition view is the primary view: the functional decomposition of the system, close to a domain model, from which the later module views grow. The Components-and-Connectors view is the key runtime view, driven by the scenario the team rates highest in value and risk — the guaranteed delivery of store events to the distribution center. The draft also carries a full interface specification for the key component, the Communications Front-End. The remaining required view, an Allocation view, and the sections that depend on it, are named and scoped here but deferred to the second draft. Every section that is not yet written says so explicitly rather than being omitted.

1.3 How the SAD Is Organized

This document follows the section order of the SEI Views and Beyond template. Section 1 is the roadmap: it explains the document's purpose and scope, the stakeholders it is aimed at, the viewpoints it uses, and how a view is documented. Section 2 explains why the architecture is what it is: the problem background and driving requirements, and the solution background, including the architectural approaches, analysis results, and requirements coverage. Section 3 presents the architecture as views, and Section 4 describes how those views relate to one another. Section 5 lists the referenced materials, and Section 6 is the directory: an index, a glossary, and an acronym list.

1.4 Stakeholder Representation

Table 1 lists the stakeholders, their concerns, and the scenarios of Section 2.1.3 that record those concerns. The last three columns are the stakeholder-to-viewpoint summary: a mark shows a viewpoint that addresses the stakeholder's concerns. The Allocation viewpoint comes in the second draft.

Stakeholder	Concerns	Scenarios	Module	C&C	Alloc.
Store manager (user)	Store inventory always readable; easy shipping orders; access only to own store.	AV-1, USE-1, SEC-3		x	
DC manager (user)	Accurate master record; clear reports.	AV-2, AV-3, PERF-3, USE-2		x	
Chain management (acquirer)	Growth to ten stores; protected inventory data.	SCAL-1, SEC-1		x	x
Store systems operator	Recovery after an outage; failover time.	AV-1, TEST-2, PORT-1		x	x
DC operator	Throughput and recovery at the DC; secure channel.	AV-3, PERF-2, SEC-2		x	x
Client developers	Subscribe over the Inventory Exchange Protocol; add client types.	MOD-1, SCAL-2, INT-2	x	x	
Supplier integrator	The information exchanged with external systems.	INT-1	x	x	
Developers	Element responsibilities, interfaces, testability.	MOD-1, MOD-2, TEST-1	x	x	
Maintainers	Effect of a change; platform moves.	MOD-2, PORT-1, TEST-2	x		x
Course instructor (evaluator)	Complete, traceable document; measurable scenarios.	All	x	x	x

Table 1: Stakeholders, concerns, and the viewpoints that address them.

1.5 Viewpoint Definitions

This draft's key view is the Components-and-Connectors view, and the C&C viewpoint is defined in full below in the form the SEI Views and Beyond template uses. The summary of which stakeholders each viewpoint serves is presented with the stakeholder matrix in Section 1.4. The module decomposition viewpoint used by the primary view (Section 3.2) follows the SEI template's module decomposition style ("Software Architecture Document Template," § Module Decomposition Viewpoint; Bass, Clements, and Kazman 10–11) and is not restated here; the Allocation viewpoint is deferred to the second draft.

1.5.1 Components-and-Connectors Viewpoint Definition

Abstract. A Components-and-Connectors view shows the system at runtime: the units that compute and hold state (components) and the pathways through which they interact (connectors). It is the view that answers how an Inventory Event travels from a store to the DC and on to the clients that need it, and what happens when the link between them fails (Bass, Clements, and Kazman 335).

Stakeholders and their concerns addressed. Store managers and store operators who need the local inventory to stay reachable during a WAN outage; DC managers who rely on the master record and the shipping orders; operators who need the acknowledgement and replay rules to be explicit; security reviewers who need the store-to-DC channels marked; and developers of new subscribing clients. Their concerns are the availability, delivery, performance, security and modifiability scenarios of Section 2.1.3.

Elements, relations, properties, and constraints. The elements are the runtime components POS Adapter, Store Agent, Store-Local Inventory, Outbox, Communications Front-End, DC Agent, Central Master Inventory,

Shipping-Order Generation, Supplier Gateway, Store-Tracking Client and Reporting Client, together with the two external systems at the boundary and the store and DC managers as actors. The relations are four connector types:

- `«pub-sub»` — the single n-ary Inventory Exchange Protocol connector, with the Communications Front-End as the broker between the publishers and the subscribers;
- `«request/reply»` — synchronous calls, such as a manager or a client querying the Central Master Inventory or a boundary element exchanging messages with an external system;
- `«data access»` — a component reading or writing its own persistent store;
- `«local call»` — an in-process hand-off between a component and its own boundary element.

The properties recorded for each element and connector are location, persistence, protocol, security, delivery semantics and throughput. The constraints are listed with the view in Section 3.1.

Language(s) to model or represent conforming views. A conforming view is drawn as a UML component diagram for its primary presentation and as UML sequence diagrams for its behavior, authored in PlantUML with the shared notation key of Section 3 (Bass, Clements, and Kazman 341; Hawker, "Architecture Documentation," slides 14–17).

Applicable evaluation and consistency criteria. A conforming view is complete and consistent when every element and connector in a diagram has an entry in the element catalog and the reverse, and when every scenario that touches the runtime has a path through the view.

Viewpoint source. The fourth edition of *Software Architecture in Practice*, Chapter 22, and the SEI Views and Beyond Software Architecture Document template (Bass, Clements, and Kazman 333–338; "Software Architecture Document Template," § Viewpoint Definitions).

1.6 How a View Is Documented

Each view in Section 3 is documented with the view-packet structure of the SEI Views and Beyond template. A view packet has a *primary presentation*, usually a diagram, that shows the view's elements and the relations among them; an *element catalog* that details the elements, their relations, their interfaces, their behavior, and any constraints; a *context diagram* that fixes the boundary of the part of the system shown; a *variability guide* that records where and when the architecture can be varied; and *architecture background* that gives the rationale for decisions whose scope is limited to the view. Every view is presented as a diagram supported by narrative, and every diagram carries a key and a caption that names the view type.

1.7 Relationship to Other SADs

Not applicable. We maintain one SAD for this system; no separate subsystem SADs are assumed by this draft. External systems are described through their boundary interfaces rather than through dependencies on other architecture documents.

1.8 Process for Updating this SAD

We maintain LaTeX and PlantUML sources in GitHub, following the SEI Views and Beyond structure. Changes use branch-based pull requests with another author's review of the source and PDF before merge. CI

compiles the PDF and runs lint checks on every push and pull request. The revision history (Section 1.1) records integrated draft and submission milestones.

We coordinate through Slack and divide the initial work into four slices: Leon Letournel owns overview, context and requirements, reviewed by Mike Andrews; Godson Umoren owns the runtime (C&C) view and the module decomposition view, reviewed by Kobe Laprade; Kobe Laprade owns the Inventory Exchange Protocol specification, reviewed by Leon Letournel; Mike Andrews owns approaches, tactics and analysis, reviewed by Godson Umoren. Later drafts extend these areas through the same review process.

2 Architecture Background

2.1 Problem Background

2.1.1 System Overview

The system tracks the inventory of a regional supermarket chain. The chain has three stores and one distribution center (DC) today, and it will grow to ten stores. The DC ships product to the stores.

Each store records its sales in real time. The point-of-sale (POS) system in the store sends each sale to the system, which turns it into an Inventory Event. A store emits up to 100 events per second. The store keeps its own copy of its inventory, the Store-Local Inventory, so that store managers can read and change it when the network to the DC is down.

The DC keeps the Central Master Inventory, the master record of the stock in every store and in the DC. The store events reach the DC through the Communications Front-End, which also sends them to a variable number of other clients. From minimum stock quantities and current sales, the DC makes shipping orders. It sends purchase and shipping orders to the external supplier/ERP system.

Products belong to product families, and a family can contain other families to any depth. Managers ask for three types of report: the lowest-stock items, the overstocked items, and the total value of a product family.

External suppliers and ERP systems are out of scope for this project. Focus on orders from stores to the DC. Don't worry about how the DC gets its products. Assume there is enough product in the DC to satisfy store order requests.

2.1.2 Goals and Context

The system has four goals:

- keep the store inventory available to the store, even when the network to the DC fails;
- keep one accurate master record of all stock, with no lost store event;
- make shipping orders and reports that managers understand with minimal training;
- let the chain add stores and clients without a change to the existing parts.

Figure 1 shows the system boundary. Inside it are the stores and the DC. Outside it are the two external systems, the POS system in each store and the supplier/ERP system, and the store and DC managers. Table 2 lists the information that we exchange with each external system.

External system	Comes in	Goes out	How often
POS system (one per store)	Sales records, through the POS Adapter.	Nothing.	Each sale; up to 100 events/s per store.
Supplier/ERP system	Goods-receipt confirmations, through the Supplier Gateway.	Purchase and shipping orders from Shipping-Order Generation.	Each order and each delivery.

per second

You seem confused. Think of the PoS system elements as each cashier scanning products a shopper purchases. A sales event results from each scan, resulting in the store inventory decreasing. When the inventory levels hit a designated minimum, the inventory tracking system creates a shipping order for the DC to send more product to the store.

Table 2: Information exchanged with each external system.

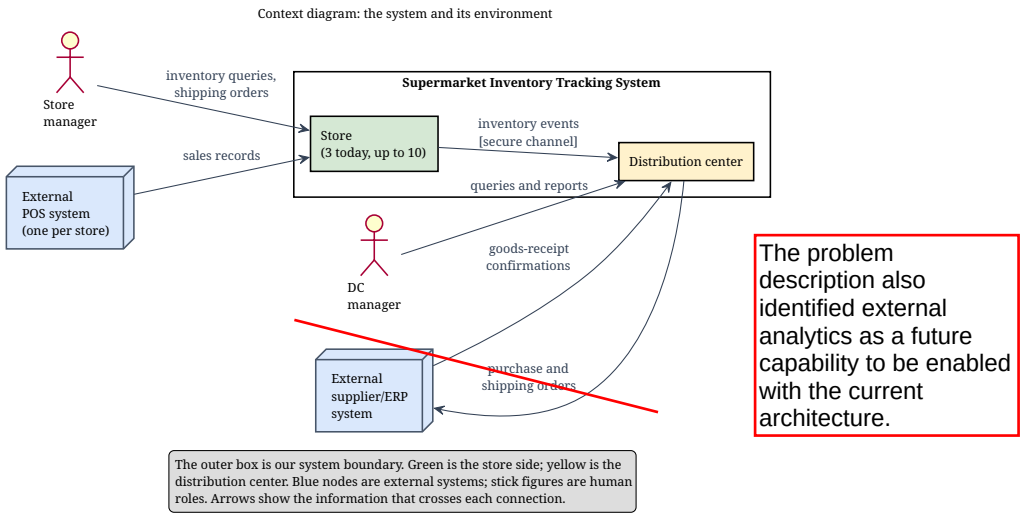


Figure 1: Components and Connectors view type: system context diagram.

2.1.3 Significant Driving Requirements

Functional requirements. The brief asks the system to:

FR-1 record sales in real time and keep a local inventory in each store, plus a master inventory that covers all store and DC stock;
From a store to the DC to replenish store shelves.
FR-2 make shipping orders from minimum stock quantities and current sales;
FR-3 provide a front-end and an exchange protocol that send alerts to the central inventory, to store tracking, and to a variable number of other clients;
Can a product be part of more than one family?
FR-4 support product families nested to any depth;
FR-5 give managers lowest-stock, overstocked-item and product-family total-value reports;
FR-6 connect to external subsystems with the exchanged information defined.

Quality-attribute requirements. The quality-attribute requirements that drive the architecture are recorded as six-part scenarios (Hawker, “Quality Attributes, Quality Scenarios, and Design Tactics,” slides 8–9). Each scenario has a frozen identifier; the full set of identifiers and their one-line titles is listed below. The brief states availability, performance, scalability, security and usability. We found the hidden requirements for modifiability, integrability, testability and portability in its variable clients, nested families and external systems. Where the brief gives no number, the response measure is our own target, which we will confirm with the stakeholders.

Scenario ID	Title
AV-1	A store loses the WAN link to the DC
AV-2	A store product event is lost between store and DC
AV-3	The Central Master Inventory process crashes at the DC
PERF-1	One store sustains its full event rate (100 events/s)
PERF-2	All ten stores at full rate reach the DC (1,000 events/s)
PERF-3	A manager report aggregates nested product families under load
SEC-1	An unauthorized actor tries to read or modify an inventory database
SEC-2	An attacker intercepts store–DC communication
SEC-3	A manager acts outside their authorized scope
SCAL-1	The chain grows from three stores to ten
SCAL-2	A new alerting client subscribes to the Communications Front-End at runtime
USE-1	A store manager produces a shipping order with minimal training
USE-2	A DC manager interprets the three report types
MOD-1	A new client type is added to receive Inventory Event alerts
MOD-2	The product-family hierarchy gains a nesting level
INT-1	An external system is integrated over the Inventory Exchange Protocol
INT-2	A variable number of alerting clients are added through the Communications Front-End
TEST-1	The guaranteed-delivery path is verified under message loss
TEST-2	Availability detection and failover timing are verified
PORT-1	A store or DC node is moved to a different platform

This is under normal operation. It will be higher when a flurry of buffered requests are made while recovering from a network outage.

Out of scope for this project.

External to what? Store to DC communications are inside the inventory tracking system (over a WAN).

Quality-attribute scenario AV-1	
Source	The wide-area network between a store and the DC (external physical infrastructure).
Stimulus	An omission fault: the store loses its network link to the DC.
Artifact	The communication channel between the Store Agent and the Communications Front-End.
Environment	Normal operation.
Response	The Store Agent keeps serving reads and updates against the Store-Local Inventory, logs the fault, and appends each outbound Inventory Event to the durable Outbox for delivery when the link returns.
Response measure	The Store-Local Inventory stays readable and updatable at least 99% of the time; this single outage lasts no more than 30 minutes; no buffered event is lost on reconnect.

Quality-attribute scenario AV-2	
Source	The WAN or the Communications Front-End (transient fault).
Stimulus	An Inventory Event or its Acknowledgement is lost between the Outbox and the Communications Front-End.
Artifact	The Outbox, the Inventory Exchange Protocol connector and the Communications Front-End.
Environment	Normal operation, 3 to 10 stores at full rate.
Response	The Outbox keeps the event and sends it again with its original ID until the Communications Front-End acknowledges it. The Communications Front-End and the Central Master Inventory discard duplicates by event ID and sequence.
Response measure	100% of store events reach the DC; 0 events lost; 0 events applied twice.

Quality-attribute scenario `AV-3`	
Source	An internal software fault at the DC.
Stimulus	The Central Master Inventory process crashes.
Artifact	The Central Master Inventory and its database.
Environment	Normal operation; stores keep publishing.
Response	The Communications Front-End keeps accepting and storing events. The process restarts, runs `subscribe`, and runs `replay-from-sequence` from its last applied sequence.
Response measure	0 events lost; the master record is current again within 30 minutes; store operation stops for 0 seconds.

You are referring to a number of components and behaviors of the system that you have not yet introduced. Try to make the requirements independent of the decomposition structures.

Quality-attribute scenario `PERF-1`	
Source	The POS system of one store.
Stimulus	Sales and stock events arrive at 100 events/s for a sustained period.
Artifact	The POS Adapter, the Store Agent, the Store-Local Inventory and the Outbox.
Environment	Normal operation, peak trading.
Response	The Store Agent applies each event locally and appends it to the Outbox.
Response measure	100 events/s sustained; each event is in the Store-Local Inventory within 1 s; the Outbox backlog does not grow.

The backlog will grow during a DC or communications outage.
What is the measure for recovery time after outage?

Quality-attribute scenario `PERF-2`	
Source	The Store Agents of ten stores.
Stimulus	All stores publish at full rate: 1,000 events/s at the DC.
Artifact	The Communications Front-End, its event store and its subscribers.
Environment	Maximum scale: ten stores and one DC.
Response	The Communications Front-End persists each event, acknowledges it, and publishes it to every subscriber.
Response measure	1,000 events/s sustained; each event acknowledged within 1 s; no queue at the DC grows.

It seems that subscribers need to acknowledge, too.

Quality-attribute scenario `PERF-3`	
Source	A DC manager.
Stimulus	Asks for the total value of a product family with nested sub-families.
Artifact	The Reporting Client and the Central Master Inventory.
Environment	1,000 events/s arrive at the DC.
Response	The Reporting Client goes through the family tree and returns the report.
Response measure	The report returns within 10 s; the Communications Front-End still sustains 1,000 events/s.

Quality-attribute scenario `SEC-1`	
Source	An unauthenticated person or process, inside or outside the network.
Stimulus	Tries to read or change a store or master inventory database.
Artifact	The Store-Local Inventory, the master inventory database and their access paths.
Environment	Normal operation.
Response	The system rejects all access without the credentials of an authorized manager or agent, and logs the attempt.
Response measure	100% of attempts rejected and logged; 0 records read or changed.

Quality-attribute scenario `SEC-2`	
Source	An attacker on the WAN.
Stimulus	Captures or changes Inventory Exchange Protocol messages between a store and the DC.
Artifact	The channel between the Outbox and the Communications Front-End.
Environment	Normal operation.
Response	TLS encrypts each message and checks its integrity. The Communications Front-End rejects a changed message and does not acknowledge it, so the Outbox sends it again.
Response measure	0 events readable by the attacker; 100% of changed messages rejected; 0 changed events applied.

Quality-attribute scenario `SEC-3`	
Source	An authenticated store manager.
Stimulus	Tries to read or change the inventory of a different store.
Artifact	The access control of the Store Agent and the Central Master Inventory.
Environment	Normal operation.
Response	The system denies the request and logs it.
Response measure	100% of out-of-scope requests denied and logged; 0 records changed.

Quality-attribute scenario `SCAL-1`	
Source	Chain management.
Stimulus	The chain adds stores, from three to ten.
Artifact	The Store Agent deployment and the Communications Front-End.
Environment	Normal operation; stores are added one at a time.
Response	We deploy one configured Store Agent for each new store. It publishes to the Communications Front-End like the existing stores.
Response measure	0 code changes to the Communications Front-End or the subscribers; `PERF-2` still holds at ten stores.

Quality-attribute scenario `SCAL-2`	
Source	A client developer.
Stimulus	A new alerting client subscribes to the Communications Front-End at runtime.
Artifact	The Communications Front-End.
Environment	Normal operation at full load.
Response	The Communications Front-End registers the subscription and sends the client each new event on its topics.
Response measure	0 Communications Front-End restarts; 0 events lost to the existing subscribers; the client receives the next published event.

Quality-attribute scenario `USE-1`	
Source	A store manager who is new to the system.
Stimulus	Wants to produce a shipping order.
Artifact	Shipping-Order Generation and its manager screen.
Environment	Normal operation.
Response	The system proposes an order from minimum stock quantities and sales. The manager checks it and confirms it.
Response measure	After at most 1 hour of training, 9 of 10 managers complete an order without help in less than 5 minutes.

Quality-attribute scenario `USE-2`	
Source	A DC manager who is new to the system.
Stimulus	Reads the lowest-stock, overstocked-item and family-value reports.
Artifact	The Reporting Client.
Environment	Normal operation.
Response	Each report shows its type, scope and units with the data.
Response measure	After at most 1 hour of training, managers answer 9 of 10 questions about the reports correctly.

Quality-attribute scenario `MOD-1`	
Source	A developer.
Stimulus	Adds a new client type that receives Inventory Event alerts.
Artifact	The new client, the Inventory Exchange Protocol and the Communications Front-End.
Environment	Design time.
Response	The developer writes the client against the Inventory Exchange Protocol and subscribes it to the Communications Front-End.
Response measure	0 changed modules outside the new client; done in at most 5 person-days.

Quality-attribute scenario `MOD-2`	
Source	A DC manager or a developer.
Stimulus	The product-family hierarchy gains one more nesting level.
Artifact	The product-family model and the Reporting Client.
Environment	Runtime, as a data change.
Response	The recursive family model accepts the new level as data; the reports include it.
Response measure	0 code changes; 100% of family-value reports correct for the new level.

Quality-attribute scenario `INT-1`	
Source	A supplier integrator.
Stimulus	Connects an external system, for example a new supplier/ERP system, over the Inventory Exchange Protocol.
Artifact	The Supplier Gateway and the Inventory Exchange Protocol.
Environment	Development time.
Response	The integrator writes an adapter that maps the external format to Inventory Exchange Protocol messages.
Response measure	Changes stay in one boundary element; done in at most 10 person-days.

Quality-attribute scenario `INT-2`	
Source	Client developers.
Stimulus	Add a variable number of alerting clients through the Communications Front-End.
Artifact	The Communications Front-End and its fan-out.
Environment	Normal operation at full load.
Response	Each client subscribes over the Inventory Exchange Protocol; the Communications Front-End sends events to all of them.
Response measure	With up to 10 subscribers, `PERF-2` still holds; 0 Communications Front-End code changes.

Quality-attribute scenario `TEST-1`	
Source	A tester.
Stimulus	Drops 10% of the Inventory Exchange Protocol messages and acknowledgements during a load test.
Artifact	The Outbox, the Communications Front-End and the Central Master Inventory.
Environment	Test environment, 100 events/s per store.
Response	The test harness controls the loss and compares the sent events with the applied events by ID and sequence.
Response measure	The test finds 100% of lost or duplicate events; one test run takes at most 1 hour.

Quality-attribute scenario `TEST-2`	
Source	A tester.
Stimulus	Stops a Store Agent or cuts the WAN to measure detection and failover.
Artifact	The Store Agent and its supervision.
Environment	Test environment.
Response	The harness records the time of the failure, the detection and the restored access.
Response measure	Recovery time measured to within 1 s; the test shows an outage of at most 30 minutes.

Quality-attribute scenario `PORT-1`	
Source	An operator or a developer.
Stimulus	Moves a store or DC node to a different operating system or hardware.
Artifact	The Store Agent or the DC services.
Environment	Maintenance window.
Response	We rebuild the node with a platform adapter and new configuration; the Inventory Exchange Protocol does not change.
Response measure	0 changes to the Inventory Exchange Protocol or other nodes; done in at most 5 person-days; all `AV` and `PERF` scenarios still pass.

Utility tree. Table 3 is our draft utility tree, in the tabular form of Bass, Clements, and Kazman, Table 19.1 (284–286; Hawker, “Utility Trees,” slides 2, 3 and 6). Each scenario has a rating of (business value, technical risk) on the book’s scale: value H is must-have, M is important but not fatal if omitted, and L is nice to have; risk H keeps the architect awake, M is concerning, and L is confident.

In a later draft, address the (H,M) scenarios, too.

The (H, H) scenarios are `AV-2` and `PERF-2`. Both are on the store-to-DC delivery path, so that path is the vertical slice of this draft. The tree focuses our design effort and is an input to the later ATAM.

2.2 Solution Background

2.2.1 Architectural Approaches

We use a POS Adapter to translate sales records for the Store Agent, which saves an inventory change and its Outbox event in one local transaction. This realises the Transactions tactic (Bass, Clements, and Kazman 61). The Outbox forwards committed events to the Communications Front-End and retains unacknowledged events for Retry (60), preserving their event IDs. Subscribers applying inventory updates must atomically save their state changes, duplicate-detection records and contiguous per-source processing progress. Repeated delivery can then have one inventory effect. Retry limits and escalation must retain undelivered records rather than discard committed changes.

The Communications Front-End persists events before returning an Acknowledgement, then distributes them to Central Master Inventory, Shipping-Order Generation, Store-Tracking Client and Reporting Client. The DC Agent publishes the DC’s own stock events; the Supplier Gateway exchanges orders and receipts with

Quality attribute	Refinement	Scenario	(V, R)
Availability	Local operation during an outage	AV-1 WAN loss at a store	(H, M)
	Guaranteed delivery	AV-2 **Lost store event**	**(H, H)**
	DC-side recovery	AV-3 Master inventory crash	(H, M)
Performance	Per-store throughput	PERF-1 One store at 100 events/s	(H, M)
	Aggregate throughput at the DC	PERF-2 **Ten stores at the DC**	**(H, H)**
	Report latency under load	PERF-3 Nested-family report under load	(M, M)
Security	Data access control	SEC-1 Unauthorized database access	(H, L)
	Channel confidentiality and integrity	SEC-2 Intercepted store–DC traffic	(H, M)
	Authorization scope	SEC-3 Manager out of scope	(M, L)
Scalability	Store growth	SCAL-1 Three to ten stores	(H, M)
	Client growth	SCAL-2 Client subscribes at runtime	(M, L)
Usability	Training effort for orders	USE-1 Shipping order, minimal training	(H, L)
	Training effort for reports	USE-2 Three report types	(H, L)
Modifiability	New client type	MOD-1 New client type	(M, L)
	Product-family structure	MOD-2 Extra nesting level	(M, M)
Integrability	External systems	INT-1 External system over the Inventory Exchange Protocol	(M, M)
	Alerting clients	INT-2 Variable alerting clients	(M, L)
Testability	Delivery under loss	TEST-1 Delivery under message loss	(M, M)
	Failover timing	TEST-2 Failover timing	(M, M)
Portability	Node platform change	PORT-1 Node moved to another platform	(L, M)

Table 3: Draft utility tree: (business value, technical risk).

Out of scope

Publish-subscribe is a pattern in the textbook, pp. 129-130

~~the external supplier/ERP system~~. Publish–subscribe realises Use an intermediary (107; 123), supporting a variable client population. The Inventory Exchange Protocol and Inventory Event definitions form a shared local standard, realising Adhere to standards (107). An acknowledgement confirms durable central acceptance, not completion by every subscriber; the central event store must retain the history needed for subscriber recovery.

Alternatives and trade-offs. We prefer local commitment followed by asynchronous delivery over requiring a successful synchronous DC call before each local update: synchronous confirmation would make store operation depend on the WAN and central service. We also prefer an Outbox over a full broker in every store, because the current local responsibility is durable forwarding, while subscription management and fan-out are centralised. These choices reduce local messaging infrastructure and duplicated routing logic, but introduce queues, retry traffic and temporary differences between local and central inventory. The delivery argument assumes surviving durable records, eventual communication recovery and sufficient storage; buffering does not establish an unconditional guarantee under permanent isolation or storage loss.

Local recovery. For the local-service outage limit, we propose supervised Store Agent restart as an initial level of Escalating restart (60–61). The restarted agent reopens the recovered Store-Local Inventory and resumes pending Outbox transmissions with their original IDs. A local snapshot is a recovery checkpoint only when committed changes after it can also be recovered; pending outgoing events alone are not a complete inventory history. Detection, restart and restoration of usable local access must together meet the 30-minute limit. Further escalation and recovery from machine or storage failure remain to be specified and validated.

Scenario tactics. The following rows cover every scenario ID. Parenthesised page numbers refer to Bass, Clements, and Kazman, fourth edition. Rows marked *proposed* or *options* describe refinements to allocate

and validate, rather than demonstrated behaviour. Repeated headers keep the table groups readable.

Use patterns, too, such as publish-subscribe.

Scenario	Tactic (4e name, page)	Realised in	Trade-off
AV-1	Graceful degradation (60)	Store Agent serves Store-Local Inventory; Outbox buffers during WAN loss.	Central state lags; storage capacity limits disconnected operation.
AV-2	Transactions (61); Retry (60)	Atomic local update/append; Communications Front-End durable acknowledgement; same-ID retries.	Extra persistence, traffic and duplicate tracking.
AV-3	Transactions (61)	Proposed atomic Central Master Inventory state/progress save; `replay-from-sequence` through Communications Front-End.	Retained history and a central recovery delay.
PERF-1	Introduce concurrency (141)	Proposed independent local processing and Outbox transmission, preserving source order.	Coordination and resource overhead; validate 100 events/s.
PERF-2	Introduce concurrency (141); Bound queue sizes (142)	Proposed concurrent central processing with explicit queue limits.	Contention; full-queue policy and recovery headroom required.
PERF-3	Increase efficiency of resource usage (141)	Proposed indexed inventory queries and arbitrary-depth family traversal.	Index storage, update overhead and query complexity.

Scenario	Tactic (4e name, page)	Realised in	Trade-off
SEC-1	Authenticate actors; Authorize actors (174)	Options for manager/agent identities, inventory access checks and restricted database accounts.	Identity infrastructure and permission administration.
SEC-2	Encrypt data (175); Verify message integrity (174); Authenticate actors (174)	TLS on store–DC legs, as the runtime view specifies; endpoint authentication and authorization mechanisms remain open.	Key/credential management and cryptographic overhead.
SEC-3	Authorize actors (174)	Proposed operation and inventory-scope checks in Store Agent and Central Master Inventory access paths.	Accurate scope assignments and consistent enforcement.
SCAL-1	Defer binding (configuration time, 124)	Proposed store identities and connection settings supplied at deployment.	Provisioning work; central throughput needs separate validation.
SCAL-2	Use an intermediary (107)	Communications Front-End registers conforming subscribers through `subscribe` at runtime.	Each client adds distribution work and possible backlog.

Scenario	Tactic (4e name, page)	Realised in	Trade-off
USE-1	Maintain task model (201)	Proposed order-task context and guidance; manager-facing entry point to assign.	Interaction-state logic; minimal-training criterion to define.
USE-2	Maintain task model (201)	Proposed Reporting Client guidance based on report type, inputs and scope.	Per-report interaction logic and user evaluation.
MOD-1	Use an intermediary (107; 123)	New client uses Inventory Exchange Protocol without changing existing publishers.	Shared-contract compatibility and extra subscriber load.
MOD-2	Encapsulate (106; 123)	Proposed shared hierarchy operations over recursive product-family relationships.	Traversal and cycle-validation complexity.
INT-1	Use an intermediary; Adhere to standards (107)	POS Adapter and Supplier Gateway map external exchanges to defined internal contracts.	Mapping maintenance; identifiers, units and meaning must agree.
INT-2	Use an intermediary (107)	Communications Front-End subscription interface serves a variable number of conforming clients.	Central dependency and fan-out resource costs.

Scenario	Tactic (4e name, page)	Realised in	Trade-off
TEST-1	Specialized interfaces (188); Sandbox (189)	Proposed isolated test instance with controlled message/acknowledgement loss and observable outcomes.	Test infrastructure; finite cases support only stated assumptions.
TEST-2	Specialized interfaces (188); Sandbox (189)	Proposed isolated test instance with failure injection and timing to usable inventory access.	Instrumentation overhead; real recovery time must be measured.
PORT-1	Encapsulate (106); Defer binding (configuration time, 124)	Proposed boundaries for persistence, process supervision and platform settings.	Platform integrations, configuration and target-platform validation.

Decisions to reconcile. We still need restart escalation and node/storage recovery, full-Outbox behaviour, and a retention/replay policy covering subscriber interruptions. Peer authentication and role/scope policy remain options. The manager-facing order interaction needs an owner without assuming a new approval rule; external order dispatch also needs a duplicate-safe handoff. Portability must cover persistence and restart supervision as well as external-system adapters. The term *failover* in TEST-2 must be aligned with the restart-based recovery being proposed.

2.2.2 Analysis Results

We derive the following estimates from the quality requirements in the project brief. These are workload and capacity calculations, not measured performance results.

Local availability. For an observation period T, 99% availability allows at most $0.01T$ downtime. Over an illustrative 30-day period, this is $30 \times 24 \times 60 \times 0.01 = 432$ minutes, or 7 hours 12 minutes. Table 4.1 uses a 90-day period and lists 21 hours 36 minutes for 99% availability (Bass, Clements, and Kazman 53). Our

observation period and downtime-accounting rules remain to be agreed. Independently, the brief permits no individual local-inventory outage longer than 30 minutes; a total budget does not override that cap.

Store buffer capacity. At 100 events/s, an illustrative 30-minute WAN interruption generates $100 \times 30 \times 60 = 180{,}000$ additional events per store. The Outbox needs capacity for that backlog, plus existing queued events, operating headroom and storage overhead. A byte allocation requires the persisted event size. The 30-minute WAN duration is a sizing assumption, not a maximum network outage established by the brief's separate local-access limit.

Central throughput. Three stores produce up to $3 \times 100 = 300$ new store events/s; ten produce $10 \times 100 = 1{,}000$. The Communications Front-End and central processing path must also account for DC events, retries, subscriber fan-out and replay traffic. To drain a backlog, effective processing capacity must exceed the ongoing arrival rate. We therefore need workload validation with recovery traffic, not only a steady 1,000 events/s input test; these calculations alone establish no achieved throughput.

2.2.3 Requirements Coverage

We trace the brief's requirements to scenarios, tactics and responsible elements below. These are design mappings, not evidence that the requirements have been demonstrated. Tactic definitions, textbook pages and decision status appear in Section 2.2.1; (P) marks a proposed refinement.

Functional responsibilities.

Requirement	Scenario	Named tactic	Responsible elements / mechanism
Real-time sales; local store inventory and a master covering store and DC stock.	`PERF-1`, `AV-1`, `AV-3`	Transactions; Retry	POS Adapter feeds Store Agent; Store-Local Inventory holds local state; Central Master Inventory consumes store and DC Agent events.
Shipping orders from minimum stock quantities and current sales.	`USE-1`	Maintain task model (P), for manager assistance	Shipping-Order Generation computes orders; a manager-facing task flow remains proposed.
Front-end, exchange protocol and alerts for central inventory, store tracking and variable additional clients.	`INT-1`, `INT-2`, `SCAL-2`, `MOD-1`	Use an intermediary; Adhere to standards	Communications Front-End and Inventory Exchange Protocol; `subscribe`/`unsubscribe` connect Central Master Inventory, Store-Tracking Client and other clients.
Arbitrarily nested product families.	`MOD-2`	Encapsulate (P)	Recursive catalog relationships and traversal behind a stable catalog interface (P); no fixed category-depth assumptions.
Lowest-stock, overstock and total family-value reports.	`PERF-3`, `USE-2`	Increase efficiency of resource usage (P); Maintain task model (P)	Reporting Client queries Central Master Inventory; query optimization and contextual report guidance are proposed.
External subsystems and explicitly defined exchanged information.	`INT-1`	Use an intermediary; Adhere to standards	POS Adapter: sales in; Supplier Gateway: orders out, receipts in; Inventory Exchange Protocol defines Inventory Event and delivery operations. External wire formats need owner confirmation.

Quality attributes and hidden requirements.

Requirement	Scenario	Named tactic	Responsible elements / mechanism
99% local inventory access; no single outage over 30 minutes.	AV-1, TEST-2	Graceful degradation; Escalating restart (P)	Store-Local Inventory remains local; Store Agent recovery uses durable state. Detection, supervision and escalation need refinement and timed validation.
100% delivery and processing of store events at the DC.	AV-2, AV-3	Transactions; Retry	Outbox retention, durable Communications Front-End acceptance, `replay-from-sequence` and atomic duplicate-safe processing by Central Master Inventory. Recovery assumptions bound the claim.
100 events/s/store; growth from three stores and one DC to ten stores and one DC.	PERF-1, PERF-2, SCAL-1	Introduce concurrency (P); Bound queue sizes (P); Defer binding (configuration time) (P)	Configured Store Agent instances; sized Outbox and central capacity for 1,000 new store events/s plus recovery traffic. Load validation remains required.
Database access limited to authorized managers and agents; scoped permissions.	SEC-1, SEC-3	Authenticate actors; Authorize actors (P)	Protected Store Agent and Central Master Inventory access paths and database accounts; identity and scope enforcement require refinement.
Secure store–DC communication.	SEC-2	Encrypt data; Verify message integrity; Authenticate actors (P)	Inventory Exchange Protocol over TLS between Outbox and Communications Front-End; endpoint authentication and authorization mechanisms remain open.
Understandable orders and reports with minimal manager training.	USE-1, USE-2	Maintain task model (P)	Proposed manager guidance around Shipping-Order Generation and Reporting Client; validate representative tasks with managers.
Hidden maintainability and testability requirements.	MOD-1, MOD-2, TEST-1, TEST-2	Use an intermediary; Encapsulate (P); Sandbox (P); Specialized interfaces (P)	Stable Communications Front-End/catalog boundaries; controlled loss/crash tests and observable Outbox, Communications Front-End and subscriber state (P).
Hidden portability requirement.	PORT-1	Encapsulate (P); Adhere to standards; Defer binding (configuration time) (P)	Inventory Exchange Protocol plus proposed platform adapters and deployment configuration for Store Agent and DC services; target-platform validation remains required.

We leave quantitative estimates to Section 2.2.2. Functional calculations remain component responsibilities: usability guidance does not itself calculate order quantities or report values. Delivery and availability claims depend on the recovery assumptions in Section 2.2.1.

2.2.4 Summary of Background Changes Reflected in Current Version

Not applicable. This is our first documented architectural baseline, so there is no earlier baseline against which to summarize changes in requirements or design context. Draft revisions are recorded in Section 1.1; subsequent baselines will summarize material background changes here.

2.3 Product Line Reuse Considerations

Not applicable. We design a single inventory-tracking product, not a family of separately configured products with managed feature variation. Varying the number of stores or subscribers is deployment and runtime variation within this product, described in Section 3.1.4.

3 Views

This section presents the architecture as views. Following the fourth edition of *Software Architecture in Practice* and the SEI Views and Beyond template, architectural views fall into three categories according to the kind of element they show (Bass, Clements, and Kazman 333; "Software Architecture Document Template," § Views):

- **Module views**, whose elements are implementation units — the system's code structure;
- **Components-and-Connectors (C&C) views**, whose elements are runtime components and the connectors through which they interact;
- **Allocation views**, which map software elements onto the environment in which they are built and run.

The assignment requires all three. This first draft delivers two of them: the module decomposition view as the primary view (its decomposition only; later module views grow from it) and the C&C view of the runtime system in full as the key view. The Allocation view is deferred to the second draft. The table below classifies each required view and records its status, so that the required structure is visible even in a draft.

Name of view	Viewtype	Module?	C&C?	Allocation?	Status
Runtime C&C view (key view)	Components-and-Connectors	No	Yes	No	Delivered in this draft (Section 3.1)
Module decomposition view (primary view)	Module	Yes	No	No	Delivered in this draft (decomposition only; Section 3.2)
Allocation view	Allocation	No	No	Yes	Draft 2

The views in this document share one notation; its key is reproduced in Figure 2.

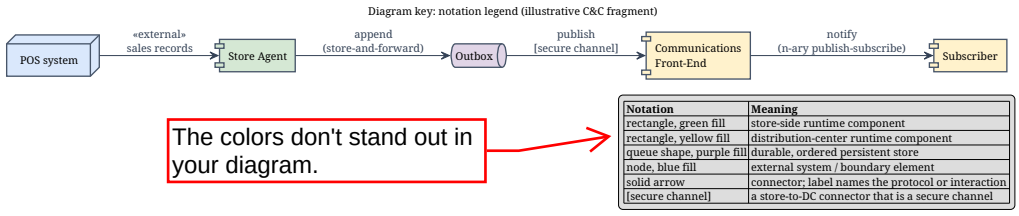


Figure 2: Diagram key: the shared notation used by the views in this document, shown on an illustrative Components-and-Connectors fragment.

3.1 Components-and-Connectors View

3.1.1 View description

This view shows the Supermarket Inventory Tracking System at runtime: the components that compute and hold state and the connectors through which they interact. It conforms to the Components-and-Connectors viewpoint of Section 1.5. Its job is to make two things readable: how an Inventory Event travels from a store's point-of-sale system through the Store Agent and its Outbox to the Communications Front-End at the DC, is acknowledged, and fans out to every subscriber; and what the system does when the link between a store and the DC fails and later recovers.

3.1.2 View packet overview

The view has one view packet, drawn at the level of the whole system. One store is drawn in full with a "one of N" multiplicity, and the DC is drawn in full. We chose a single system-level packet because the stores are identical, so a separate packet per store would repeat the same picture, and because the interactions worth showing are the store-to-DC event flow and the fan-out at the DC, both of which are visible in one diagram. The packet is therefore complete for the runtime concerns of this draft and does not duplicate itself.

3.1.3 Architecture background

This background covers only decisions whose scope is this view. The choice of the publish–subscribe pattern itself, and the alternatives we rejected (synchronous client–server and federated per-store brokers), are argued once in Section 2.2.1 and are not repeated here.

We draw the Inventory Exchange Protocol connector as a single n-ary publish–subscribe connector rather than as a mesh of point-to-point links. A publish–subscribe connector is naturally n-ary: one broker joins many publishers to many subscribers (Bass, Clements, and Kazman 335). Drawing it once, centered on the Communications Front-End broker, keeps the picture honest about what is one connector, and it stays readable as clients are added, which is the behavior the modifiability and scalability scenarios ask for.

We document two behaviors because they are the two the runtime concerns turn on. The normal path (publish, acknowledge, fan-out) shows how delivery, throughput and the secure channel work on a good day. The outage path (buffer, reconnect, replay) shows how the system meets its availability and delivery guarantees on a bad one. Between them they exercise every runtime scenario that touches this view.

3.1.4 Variability mechanisms

The view has two variation points, both exercised at run time.

- *Number of stores.* The chain starts at three stores and is designed to reach ten (`SCAL-1`). Each store is an instance of the Store boundary in the primary presentation, so adding one adds an instance and a publisher on the Inventory Exchange Protocol connector; nothing else in the view changes.
- *Number and kind of subscribers.* A client subscribes to the Communications Front-End at run time through the Inventory Exchange Protocol, so a new kind of subscriber is added without changing the Communications Front-End (`MOD-1`, `SCAL-2`, `INT-2`). Topics are named by source and product family, and the product-family hierarchy can gain a nesting level (`MOD-2`) without a change to the connector.

3.1.5 Primary presentation

Figure 3 is the primary presentation. The store boundary holds the POS Adapter, the Store Agent, the Store-Local Inventory and the Outbox; the external POS system sits outside it. The DC boundary holds the Communications Front-End and its event store, the DC Agent, the Central Master Inventory and its database, Shipping-Order Generation, the Supplier Gateway, a Store-Tracking Client and a Reporting Client; the external supplier/ERP system sits outside it. The store and DC managers appear as actors who query the system. The connectors and the notation are read from the figure's key.

3.1.6 Element catalog

The catalog describes each element and connector shown in the primary presentation. The interfaces part of the catalog is the Interfaces subsection (Section 3.1.7), which follows.

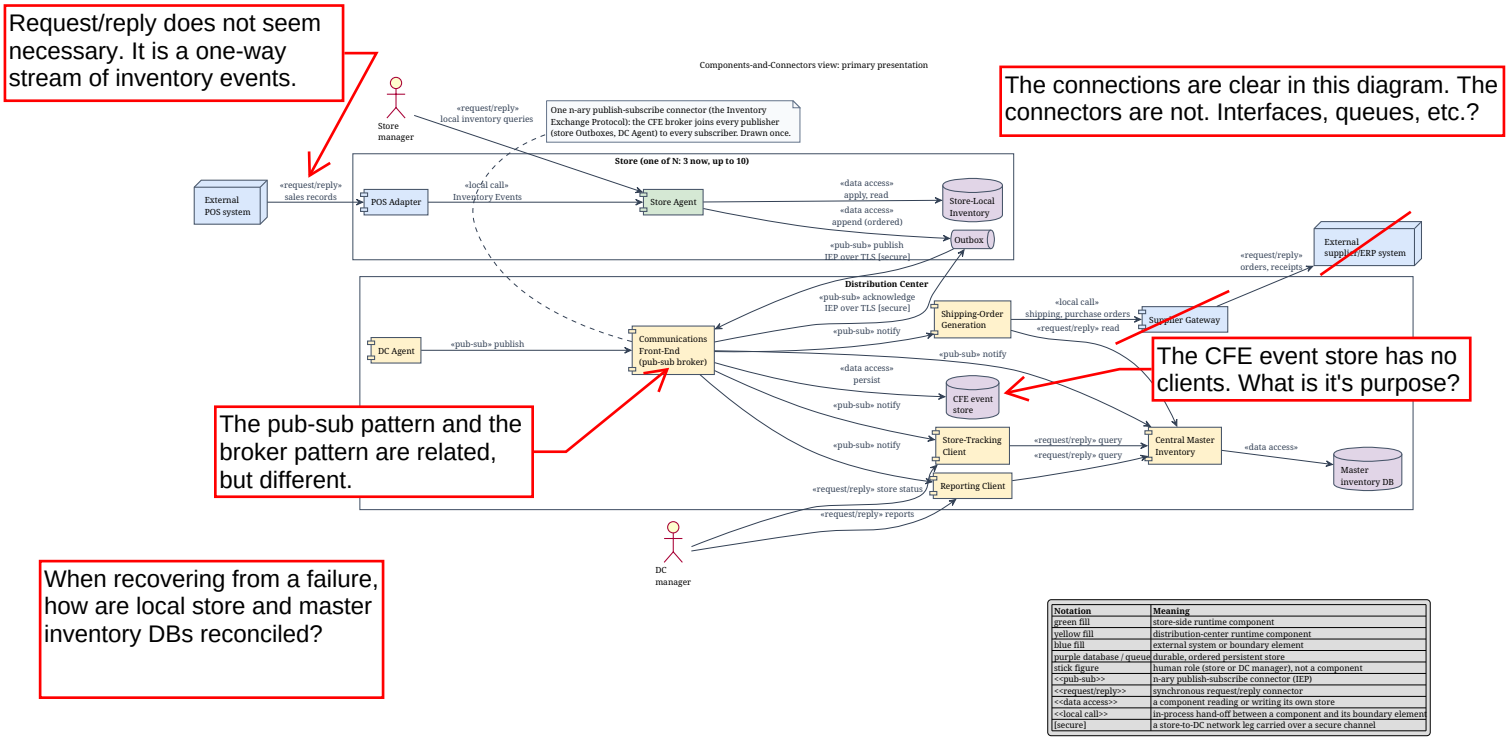


Figure 3: Components and Connectors view — primary presentation.

Elements. Table 4 lists the components. Location, persistent state and the quality properties that matter for each element are the properties named in the viewpoint (Section 1.5). The two internal stores in the figure, the CFE event store and the master inventory database, appear as their own rows, so every component box in the figure has a row and every row is a box.

Human roles (not components). The store manager and the DC manager appear as actors in the figure, not as rows above: the store manager queries its Store Agent for local inventory, and the DC manager queries the Reporting Client and the Store-Tracking Client for reports and store status.

Relations. Table 5 lists the connectors, grouped by the four stereotyped types the viewpoint defines: `«pub-sub»`, `«request/reply»`, `«data access»` and `«local call»`. The two store-to-DC legs of the Inventory Exchange Protocol connector, the publish leg and the acknowledgement leg, are the only network legs in the view, and both run over a secure channel (`SEC-2`); the figure marks them `[secure]`. Every other connector is local to a store or to the DC.

Behavior. Figures 4 and 5 document the two behaviors.

Figure 4 is the normal path. A sale reaches the POS Adapter, which hands the Store Agent an Inventory Event. In one local step the Store Agent applies the event to the Store-Local Inventory and appends it to the Outbox, so a store keeps reading and writing its own inventory without waiting on the WAN. The Outbox then runs `publish` to the Communications Front-End over the secure channel; the Communications Front-End persists the event, returns an `acknowledge` to the Outbox, and runs the fan-out, publishing by topic to every subscriber, which stays idempotent by event ID. This path answers the per-store rate of `PERF-1` and, at the DC, the aggregate of `PERF-2`; the acknowledgement answers `AV-2`; and the secure publish and acknowledge legs answer `SEC-2`.

Figure 5 is the outage path. While the WAN link is down the store keeps applying events locally and the

Element	Responsibility	Location	Persistent state	Key quality properties
POS Adapter	Turns the external POS system's sales records into Inventory Events.	Store (boundary)	None	Integration (`INT-1`)
Store Agent	Applies each event to the Store-Local Inventory and Outbox in one step; forwards to the Communications Front-End; serves the store manager's local inventory queries; restarts from the Outbox and a snapshot.	Store	Via its stores	Availability, recovery (`AV-1`)
Store-Local Inventory	The store's inventory copy, readable and updatable with no WAN.	Store	Yes (database)	99% local availability (`AV-1`)
Outbox	Durable, ordered queue holding events until the Communications Front-End acknowledges them; keeps the last acknowledged sequence number and replays unacknowledged events on reconnect.	Store	Yes (queue)	Delivery (`AV-2`); buffers a 30-min outage
Communications Front-End	Pub-sub broker: persists each event, acknowledges it to the publishing Outbox, publishes by topic to every subscriber.	DC	Yes (event store)	100% delivery (`AV-2`); 1,000 events/s (`PERF-2`)
DC Agent	Publishes the DC's own events through the Communications Front-End.	DC	None	Master covers DC stock
Central Master Inventory	Authoritative record of all store and DC stock, built from every event. After a crash it catches up by running `subscribe` to the Communications Front-End and `replay-from-sequence` from the CFE event store at its last applied sequence.	DC	Yes (database)	Recovery (`AV-3`); access (`SEC-1`)
Shipping-Order Generation	Computes shipping orders and sends them via the Supplier Gateway.	DC	None	Report usability (`USE-1`)
Supplier Gateway	Exchanges orders and receipts with the external supplier/ERP system.	DC (boundary)	None	Integration (`INT-1`)
Store-Tracking Client	Subscriber; shows a store's inventory and answers the DC manager's store-status queries.	DC	None	Variable client (`SCAL-2`, `INT-2`)
Reporting Client	Subscriber; answers the DC manager's report queries.	DC	None	Report load (`PERF-3`, `USE-2`)
CFE event store	Durable log of every acknowledged event; source for replay and fan-out.	DC	Yes	Replay (`AV-2`, `TEST-1`)
Master inventory database	Holds the stock levels kept by the Central Master Inventory.	DC	Yes	Access control (`SEC-1`)
External POS system	Store point-of-sale; emits sales records. Shown for context.	External (per store)	—	Integration (`INT-1`)
External supplier/ERP system	Receives orders; returns goods-receipt confirmations. Shown for context.	External (at DC)	—	Integration (`INT-1`)

Table 4: Element catalog — components of the Components and Connectors view.

Connector	Type	Endpoints	Protocol	Secure	Delivery semantics
Inventory Exchange Protocol bus	pub-sub (n-ary)	Publishers: store Outboxes (×N), DC Agent; broker: Communications Front-End; subscribers: Central Master Inventory, Shipping-Order Generation, Store-Tracking Client, Reporting Client	Inventory Exchange Protocol over TLS on the store-to-DC legs	Yes (store–DC)	At-least-once, ordered per source, idempotent by event ID; acknowledged to the publishing Outbox, then replayable
Master-inventory query	request/ reply	Reporting Client, Store-Tracking Client and Shipping-Order Generation to the Central Master Inventory	Internal query API	In DC	Synchronous, read-only
Manager query	request/ reply	Store manager to the Store Agent; DC manager to the Reporting Client and Store-Tracking Client	Internal query API	In store, in DC	Synchronous, read-only
POS feed	request/ reply	External POS system to the POS Adapter	External POS protocol (to be confirmed with the external system owner)	Per owner	Request/reply; the adapter converts replies into Inventory Events
Supplier exchange	request/ reply	Supplier Gateway to the external supplier/ERP system	External supplier/ERP protocol (to be confirmed)	Per owner	Orders out; goods-receipt confirmations back
Store-side store access	data access	Store Agent to the Store-Local Inventory and to the Outbox	Local	N/A	Durable, ordered append; local read and update
DC-side store access	data access	Communications Front-End to its event store; Central Master Inventory to its database	Local	N/A	Durable persistence
Boundary hand-off	local call	POS Adapter to the Store Agent; Shipping-Order Generation to the Supplier Gateway	In-process	N/A	In-process call between a component and its boundary element

Table 5: Element catalog — connectors of the Components and Connectors view.

Outbox buffers them in order, so local inventory stays reachable (`AV-1`). On reconnect the Outbox runs `replay-from-sequence` from the last acknowledged sequence number and, if a larger gap is found, `resync`; the Communications Front-End acknowledges each event and the subscribers ignore duplicates by event ID, so no event is lost (`AV-2`, `TEST-1`). The Outbox is sized for a full 30-minute outage at the per-store rate; the sizing is worked in Section 2.2.2.

The same replay mechanism covers `AV-3`: after a Central Master Inventory crash it restarts, runs `subscribe` to the Communications Front-End, and runs `replay-from-sequence` from its last applied sequence number against the CFE event store; because subscribers are idempotent by event ID, any duplicates are harmless. The replay and resync semantics (sequence numbers and gap handling) are specified in the interface specification, Section 3.1.7.

Constraints.

- There is one Communications Front-End per DC.
- The Outbox preserves order per source: events leave a store in their source sequence.
- Every subscriber is idempotent by event ID.
- A store never blocks on the WAN; local reads and writes continue during an outage.
- An Inventory Event is immutable once acknowledged.

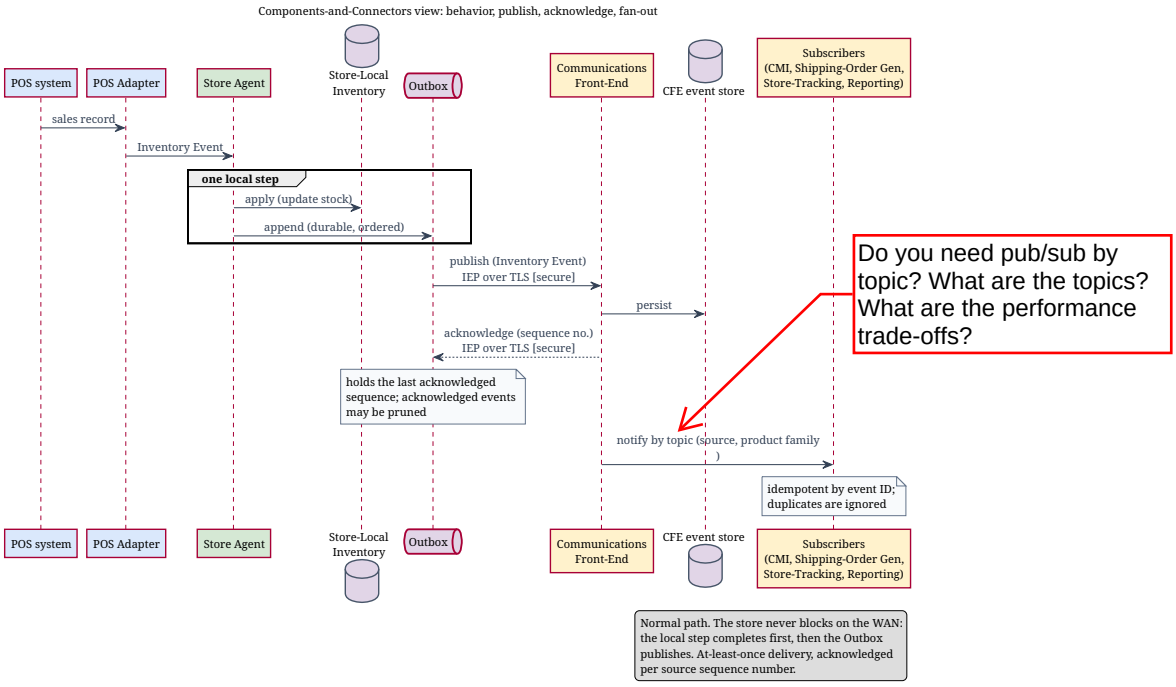


Figure 4: Components and Connectors view — behavior: publish, acknowledge, and fan-out.

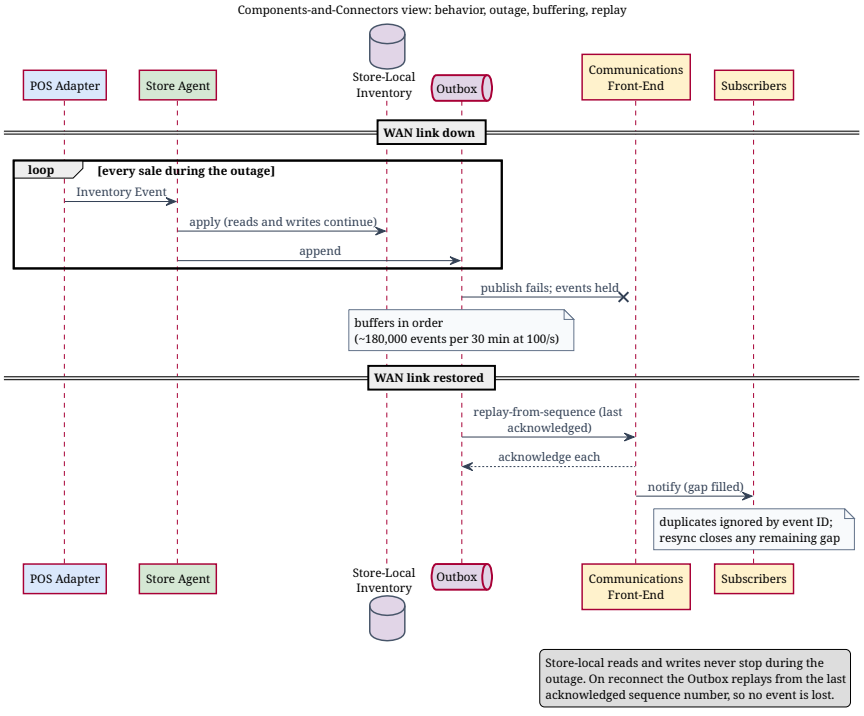


Figure 5: Components and Connectors view — behavior: outage, buffering, and replay.

The number of stores and the number of subscribers are variation points (Section 3.1.4), not constraints.

3.1.7 Interfaces

The one interface this view needs specified is the interface of the key component, the Communications Front-End, which is the Inventory Exchange Protocol. It is specified in its own file, input here so that Slice 3 can author it independently:

Inventory Exchange Protocol interface specification.

Identity, purpose, and provided/required sides. We define version 1.0 of the Inventory Exchange Protocol to connect store and DC publishers to runtime subscribers through the Communications Front-End. The provided side comprises the six operations below; the required side comprises durable storage, a subscriber registry, secure channels, a product catalog and clocks (Bass, Clements, and Kazman 230). Publishers retain unacknowledged events and subscribers process them idempotently.

Data types and topic naming. We use case-sensitive fields and nonempty identifiers compared exactly; the wire encoding preserves their types.

Inventory Event
`eventId`: globally unique identifier, unchanged on retry; `source`: `store/<storeId>` or `dc/<dcId>`; `product`: stable product identifier; `quantityDelta`: nonzero signed 64-bit stock-unit delta (positive adds); `timestamp`: UTC `YYYY-MM-DDThh:mm:ss.sssZ`; `sequence`: persistent positive 64-bit counter, consecutive per source from 1. Retries preserve all fields; sequence, not timestamp, determines order.

Acknowledgement
`eventId`, `source`, `sequence`: accepted event; `acceptedThrough`: highest contiguous persisted source sequence, not subscriber progress.

Subscription
`subscriberId`: stable authenticated client ID; `subscriptionId`: identifier assigned by the Communications Front-End; `sourceFilter`: an exact source or `*`; and `familyFilter`: an exact product-family path, that path followed by `/**` for itself and all descendants, or `*` for all families. Recovery positions are per subscription and source.

We name topics `inventory/<source>/family/<familyPath>`. Their segments contain letters, digits, hyphens or underscores, separated by slashes. The Communications Front-End resolves catalog families; both filters and authorization must match. `food/**` includes `food/dairy`, not `foodstuff`.

Resources and operations. We specify each operation through its syntax, effects and errors (Bass, Clements, and Kazman 219). All requests require authenticated, authorized version-1.0 sessions; `?` means optional. Common errors: `INVALID` (fields/version), `UNAUTHORIZED` (identity/scope), `UNAVAILABLE`, `BUSY(retryAfterMs)` (positive millisecond delay), `TIMEOUT` (uncertain outcome).

`publish (event)`
Pre/meaning: Submit an authorized source's next event. *Post:* Persist event and identity/sequence, then `acknowledge` and fan-out; identical retries create no extra entry. *Errors:* Common errors; `OUT_OF_ORDER` (returns expected sequence) for gaps, `CONFLICT` for reused IDs/sequences with changed content, `UNKNOWN_PRODUCT`. Rejection never advances the source position.

`acknowledge (acknowledgement)`
Pre/meaning: Broker response confirming durable acceptance. *Post:* The publisher (the store's Outbox,

or the DC Agent for its own source) saves the maximum of its position and `acceptedThrough` before pruning; subscriber cursors are unchanged. *Errors:* Reject malformed/mismatched or unauthenticated confirmations as `INVALID` or `UNAUTHORIZED` without pruning; lost responses cause retries.

`subscribe (subscriberId, sourceFilter, familyFilter)`
Pre/meaning: Provisioned client opens a receive channel with authorized filters. *Post:* Persist/return subscription and `startAfter`, the per-source accepted positions (zero if empty). Live events follow these positions; history requires `replay-from-sequence`. Identical active requests return the same record/positions. *Errors:* Common errors; `UNKNOWN_SUBSCRIBER`; rejection creates nothing.

`unsubscribe (subscriberId, subscriptionId)`
Pre/meaning: Owner removes its subscription. *Post:* Persist removal; stop scheduling deliveries, allowing in-flight messages. Owner retries succeed while removal records exist; logs remain unchanged. *Errors:* Common errors; `UNKNOWN_SUBSCRIPTION` without active/removal record.

`replay-from-sequence (role, source, afterSequence, subscriptionId?, events?)`
Pre/meaning: Nonnegative exclusive cursor. For `publisher`, send ordered retained events after the publisher's saved position; for `subscriber`, identify an owned active subscription, omit events and use its durable cursor. *Post:* Publisher events follow `publish` individually; partial failure identifies the first failed event. Subscriber replay returns matching events in source order through a captured high-water mark, then `scannedThrough` (that mark); live delivery follows without gaps. Advance past filtered-out positions only after processing returned events. Retries may duplicate delivery. *Errors:* Common errors and those of `publish`; `UNKNOWN_SUBSCRIPTION`, `INVALID_CURSOR` (subscriber ahead), `HISTORY_EXPIRED`. Publisher-ahead positions require `resync` rather than skipping.

`resync (role, source, lastKnownSequence, subscriptionId?)`
Pre/meaning: Authorized publisher/subscriber reconciles a nonnegative position; subscribers identify an owned active subscription. *Post:* Return `acceptedThrough`, `earliestRetained` (first replayable sequence; accepted position plus one for an empty log), and recovery action. Publishers resend after broker position; subscribers replay after their cursor. No stock/sequence resets. *Errors:* Common errors; `UNKNOWN_SUBSCRIPTION`, `INVALID_CURSOR` (subscriber ahead), `RECOVERY_REQUIRED` with bounds for missing history. Restore a consistent snapshot/backup and positions before replay; reconciliation cannot reconstruct missing events.

Shared semantics and restrictions. We restore writable logs, identity records, source positions and the registry before serving requests. Each source has one logical publisher: its Store Agent through the Outbox, or the DC Agent; failover prevents concurrent sequence allocation. Every request and delivery is authorized. Local inventory, sequence and Outbox append commit atomically. We preserve order per source, not globally; subscribers atomically save changes, deduplication and processing cursors. Identity records survive restarts throughout retry/replay history; unverifiable old events require recovery.

Why prune?

Each publisher owns its acknowledged position (the Outbox for a store, the DC Agent for the DC), saved before pruning. Clients subscribe before receiving and recover using separate subscriber cursors. Transient errors are retried unchanged with capped exponential backoff/jitter, respecting `retryAfterMs`; unacknowledged events are never discarded. Invalid requests require correction; gaps use `replay-from-sequence` or `resync` (Bass, Clements, and Kazman 228). We bound batches and queues, returning `BUSY` before admission, and isolate slow subscribers for log-based catch-up. Offline updates continue until Outbox exhaustion, which rejects updates atomically. Expired history never silently advances cursors.

Variability, limits, and quality properties. We design for the required 100 events/s/store for ten stores (1,000/s), with additional capacity for DC traffic, retries and fan-out. Clients use `subscribe` and `unsubscribe` at runtime without restarts or publisher changes. Each deployment declares tested client/queue

limits and returns BUSY at capacity. We propose, without claiming validation, 2,000 store events/s during catch-up, 1–100-event replay batches and at least 60 minutes of replay/identity history, extended for outage/catch-up headroom. Our proposed 95th-percentile targets are 1 second to acknowledge and 2 seconds to first delivery, measured from transmission at live load with healthy infrastructure, no backlog and within client limits; application processing is excluded. We provide eventual, at-least-once delivery assuming connectivity returns, retries continue and capacity/storage remain available. Beyond retention, recovery needs consistent snapshots/backups; permanent loss of all copies is unrecoverable. We leave system sizing to Section 2.2.2.

Element requirements. The Communications Front-End requires storage that atomically commits events, identities and positions before acknowledge and supports consistent backup/restore. The durable registry retains owners, filters and start/removal records; subscribers retain atomic checkpoints. The environment supplies catalog routing, TLS on the store–DC legs, peer authentication of publishers and subscribers (mechanism still open), credentials, revocation and source/topic permissions. We restrict database access; read access and wildcards grant no extra publishing scope. UTC timestamps and monotonic timeout clocks keep recovery independent of clock adjustments.

Rationale and examples. We choose asynchronous queues for unreliable distributed links (Bass, Clements, and Kazman 224). Buffering keeps updates independent of the WAN; acknowledging persistence keeps slow subscribers from delaying publishers. We use IDs for safe retries and topics to decouple clients. We accept eventual consistency and durable duplicate tracking, and must validate deployment limits and agree on wire binding.

Normal: After a sale, the Store Agent atomically updates stock and appends e41 (store/17, milk-1, delta −1, 2026-10-15T18:00:00.000Z, sequence 41). The Outbox sends publish; the Communications Front-End persists, sends acknowledge for e41 through 41, then fans out to matching subscriptions. The Outbox saves 41 before pruning; subscribers apply once despite duplicates.

Outage: We follow the same source through WAN failure. The Outbox buffers e42,e43 while local updates continue. On authenticated reconnect, it sends replay-from-sequence: publisher role, source store/17, saved cursor 41, events [e42,e43], no subscription ID. The Communications Front-End persists, acknowledges and fans out each event; the Outbox saves progress before pruning. Subscriber recovery uses its own cursor/subscription ID without an event batch; resync reconciles missing history. Identical retries are deduplicated.

Interface scenario coverage. We map the scenarios as follows. AV-1, AV-2: buffering, publish, acknowledge, publisher replay-from-sequence; AV-3: subscriber replay-from-sequence, resync, atomic cursors. PERF-1, PERF-2, SCAL-1: capacity/backpressure; SCAL-2, MOD-1, INT-2: runtime subscribe and unsubscribe; MOD-2: hierarchical topics. INT-1: adapter message/operation contracts. SEC-1: restricted storage; SEC-2: secure channels; SEC-3: source/topic authorization. TEST-1: IDs/acknowledgements and replay-from-sequence; TEST-2: timeouts and resync, plus deployment failover tests; PORT-1: portable contracts/state and agreed wire binding. PERF-3, USE-1, USE-2 concern reporting/UI outcomes, not protocol guarantees. We record system coverage in Section 2.2.3.

3.1.8 Context diagram

Figure 6 fixes the scope of this view packet. The shaded box is the whole system of the primary presentation, the store systems and the DC system taken as one. Outside it are the two external systems and the store and

DC managers, shown with the interactions the view cares about: sales records in, orders and confirmations exchanged with the supplier/ERP system, store inventory views out to the store manager, and master inventory and reports out to the DC manager. The system-level context, with the exchanged-information table, is in Section 2.1.2.

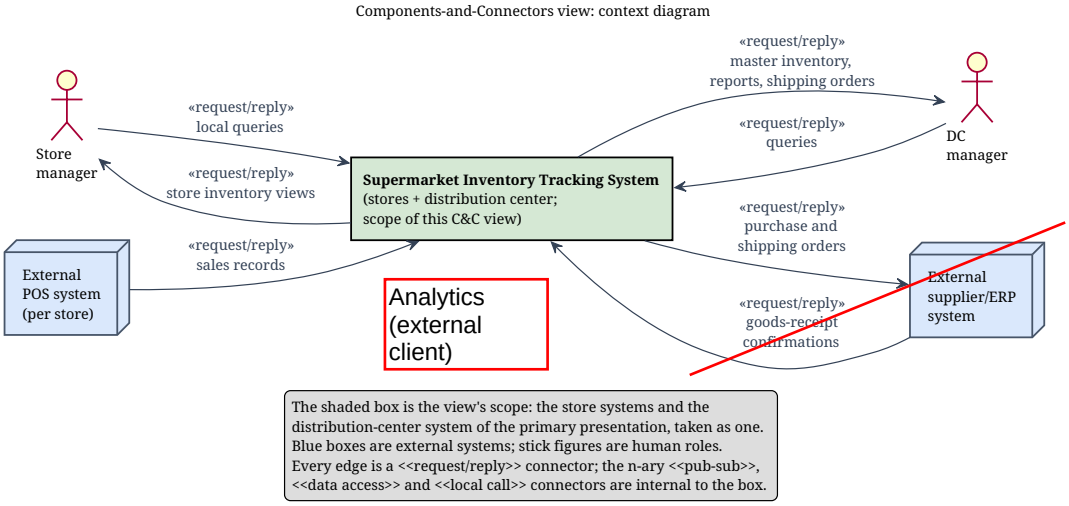


Figure 6: Components and Connectors view — context diagram.

3.1.9 Related view packets

The module decomposition view (Section 3.2) delivered in this draft shows the implementation units that realize these runtime components; the mapping between the two views is recorded in Section 4, where no inconsistencies were found. The Allocation view, which will show the nodes these components run on, is deferred to the second draft and will map onto the element names fixed here.

3.2 Module Decomposition View

3.2.1 View description

This is the primary view of the architecture: the code structure as a hierarchy of modules related by is-part-of, close to a domain model, from which the runtime components of the key view grow. It conforms to the module decomposition viewpoint of the SEI Views and Beyond template, whose element type is the module and whose relation is is-part-of (“Software Architecture Document Template,” § Module Decomposition Viewpoint; Bass, Clements, and Kazman 10–11). It shows the units a developer works in — the Store Subsystem, the Communications Front-End, the Distribution Center Subsystem and a Common module of shared code — and which runtime component each realises, so the code and runtime structures can be read against each other.

3.2.2 Primary presentation

Figure 7 is a package diagram of the four top-level modules and their sub-modules. Nesting is the is-part-of relation and the dashed arrows are the uses relation; the notation is read from the figure’s key.

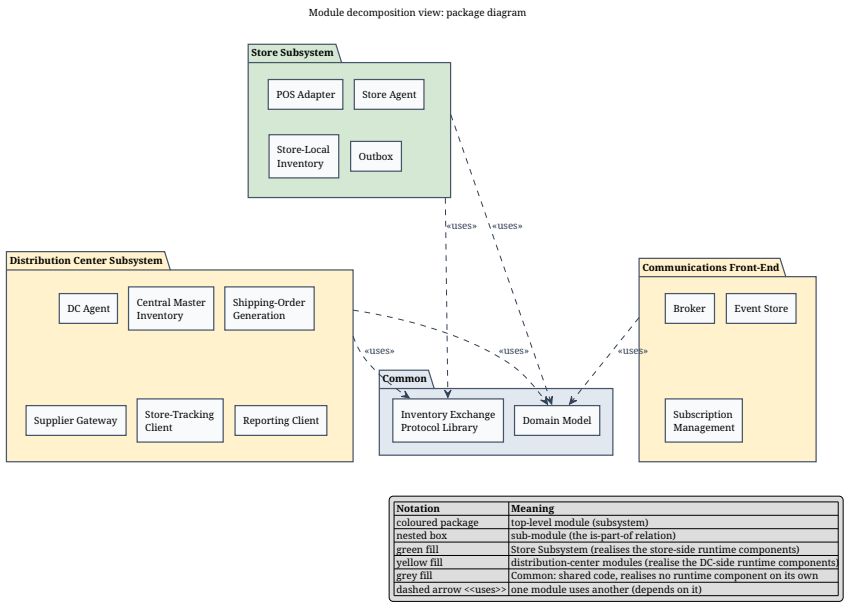


Figure 7: Module decomposition view — primary presentation.

3.2.3 Element catalog

Elements. Table 6 lists the four top-level modules and the runtime components of the Components-and-Connectors view (Table 4) each realises. Every runtime component and data store is realised by exactly one module; the Common module realises none, being shared code rather than a running element.

Module	Responsibility	Runtime components realised
Store Subsystem	Code that runs in a store: converts point-of-sale records into Inventory Events, keeps the store's own inventory copy, and buffers events for guaranteed, ordered delivery to the DC.	POS Adapter, Store Agent, Store-Local Inventory, Outbox
Communications Front-End	The broker code at the DC: persists each event, acknowledges it, manages subscriptions, and fans out by topic. Sub-modules: Broker, Event Store, Subscription Management.	Communications Front-End, CFE event store
Distribution Center Subsystem	The DC-side application code: the master record, shipping-order computation, the supplier exchange, and the subscriber clients.	DC Agent, Central Master Inventory, Shipping-Order Generation, Supplier Gateway, Store-Tracking Client, Reporting Client, Master inventory database
Common	Shared code used by the other three modules: the domain types (Figure 8) and the Inventory Exchange Protocol library.	None

Table 6: Element catalog — modules of the module decomposition view and the runtime components each realises.

Domain model. The Common module's Domain Model is decomposed in Figure 8, a class diagram of the shared domain types: a product family is a composite that nests products and sub-families to an unbounded depth; an Inventory Event carries its source, product, sequence and quantity delta; an InventoryLevel is the stock of one product at one store or DC; a ShippingOrder carries lines of product and quantity from the DC to a store; and a report is one of the three the brief names — lowest stock, overstocked items, and the total value of a product family. Every association carries multiplicities.

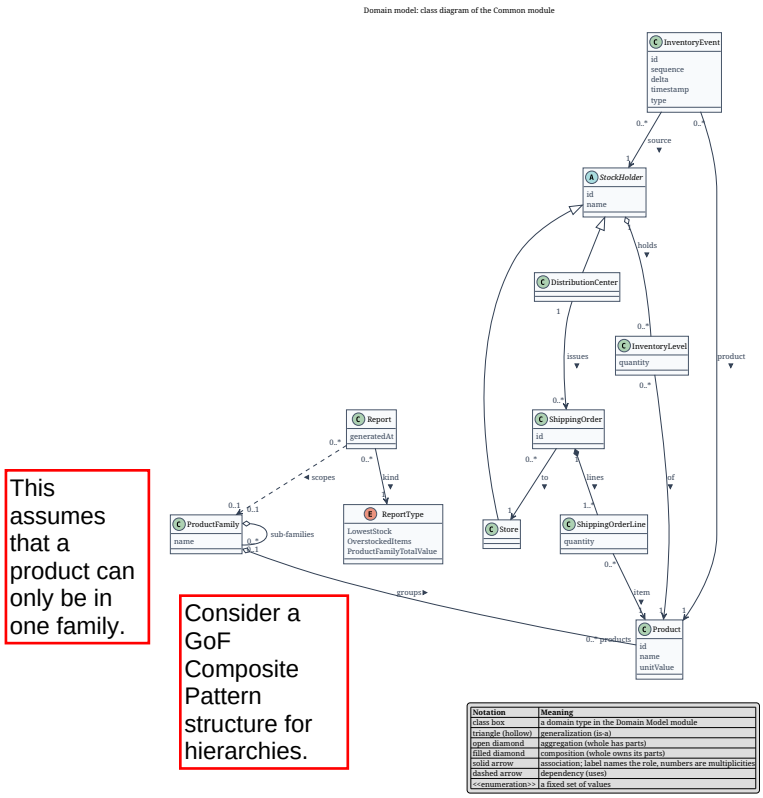


Figure 8: Module decomposition view — the Common module's Domain Model.

Relations. *Is-part-of* is the nesting in Figure 7: each sub-module is part of its subsystem. *Uses* is the five dashed arrows: the Store Subsystem, the Communications Front-End and the Distribution Center Subsystem each use the Common Domain Model, and the Store and Distribution Center subsystems additionally use the Inventory Exchange Protocol library. No other module-to-module dependency exists.

Interfaces. The only interface a module view fixes is the provided interface of the Inventory Exchange Protocol library, the full specification of Section 3.1.7; the other shared contract is the set of domain types in Figure 8.

Behavior. A module view records no runtime behavior; the event flow and the outage-and-replay behavior are documented in the Components-and-Connectors view (Section 3.1).

Constraints.

- The Common module depends on none of the subsystems; every dependency points into Common, never out of it.
- The Store Subsystem does not depend on the Distribution Center Subsystem (only on Common), so a store can be built, tested and deployed on its own.
- Each runtime component of the Components-and-Connectors view is realised by exactly one module.

3.2.4 Context diagram

The module view shares the system boundary of the key view, fixed by the context diagram of the Components-and-Connectors view (Figure 6) and not repeated here; the external point-of-sale and supplier/ERP systems are reached only through the POS Adapter and Supplier Gateway modules.

3.2.5 Variability mechanisms

- *A new client type:* a new sub-module of the Distribution Center Subsystem that uses the Inventory Exchange Protocol library, with no change to existing modules (`MOD-1`, `INT-2`).
- *Deeper product families:* nesting depth lives in the composite in the domain model, so a new level is a data change, not a module change (`MOD-2`).
- *More stores:* not a module concern — every store is the same Store Subsystem, so the chain grows by instances, not modules (`SCAL-1`).
- *A new platform:* platform-specific code is confined to the adapter sub-modules (the POS Adapter and Supplier Gateway), so a port touches only those (`PORT-1`).

3.2.6 Architecture background

This decomposition is organised for modifiability, the quality the decomposition structure most directly determines (Bass, Clements, and Kazman 10), using three coupling-reducing tactics (Bass, Clements, and Kazman 123–124): *encapsulate* (each subsystem hides its internals and platform-specific code is confined to the adapters), *use an intermediary* (the Inventory Exchange Protocol library carries events between the Store and Distribution Center subsystems, so neither depends on the other), and *restrict dependencies* (dependencies run one way, into Common, so a store builds and deploys alone and a new client is added without touching existing modules).

3.2.7 Related view packets

This primary view maps onto the Components-and-Connectors key view (Section 3.1); the module-to-component mapping is collected in Section 4. The Allocation view, deferred to the second draft, will map these modules and components onto the nodes they are built and run on.

4 Relations Among Views

This draft contains two views: the module decomposition view — the primary view, Section 3.2 — and the Components-and-Connectors view — the key runtime view, Section 3.1. The two are consistent. Every runtime component of the C&C view is realised by exactly one module of the module view, as Table 7 records. The module view adds one module, Common, that is implementation-only — the shared domain types and the Inventory Exchange Protocol library — and so has no runtime component of its own. We found no inconsistency between the two views.

The Allocation view, deferred to the second draft, will extend this mapping by placing each module and component on the node it is built and run on, and we will record any inconsistency it reveals here.

5 Referenced Materials

The sources cited in this document are listed below in MLA style.

Module (module view)	Runtime components it realises (C&C view)
Store Subsystem	POS Adapter, Store Agent, Store-Local Inventory, Outbox
Communications Front-End	Communications Front-End, CFE event store
Distribution Center Subsystem	DC Agent, Central Master Inventory, Shipping-Order Generation, Supplier Gateway, Store-Tracking Client, Reporting Client, Master inventory database
Common	None — shared code used by the other three modules

Table 7: Module-to-runtime-component mapping between the module decomposition view and the Components-and-Connectors view.

Bass, Len, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. 4th ed., Addison-Wesley, 2022.

"Software Architecture Document Template." Software Engineering Institute, Carnegie Mellon University. Based on Clements, Paul, et al., *Documenting Software Architectures: Views and Beyond*, Addison-Wesley.

Kruchten, Philippe. "Architectural Blueprints—The '4+1' View Model of Software Architecture." *IEEE Software*, vol. 12, no. 6, Nov. 1995, pp. 42–50.

ISO/IEC/IEEE 42010:2011, Systems and Software Engineering — Architecture Description. 1st ed., ISO/IEC/IEEE, 2011.

Hawker, J. Scott. "Architecture Documentation." SWEN-755 Software Architecture, Rochester Institute of Technology. Lecture slides.

Hawker, J. Scott. "Structures and Views, 4+1 Views." SWEN-755 Software Architecture, Rochester Institute of Technology. Lecture slides.

Hawker, J. Scott. "Quality Attributes, Quality Scenarios, and Design Tactics." SWEN-755 Software Architecture, Rochester Institute of Technology. Lecture slides.

Hawker, J. Scott. "Chapter 5: Availability." SWEN-755 Software Architecture, Rochester Institute of Technology. Lecture slides.

Hawker, J. Scott. "Utility Trees." SWEN-755 Software Architecture, Rochester Institute of Technology. Lecture slides.

Hawker, J. Scott. "Security." SWEN-755 Software Architecture, Rochester Institute of Technology. Lecture slides.

6 Directory

6.1 Index

Not applicable. For this first draft, the table of contents, section cross-references and glossary provide navigation without a separate alphabetical index. We will reconsider an index if the document grows enough to make these aids insufficient.

6.2 Glossary

Store Agent
The runtime component in each store that receives inventory events from the point-of-sale system (through

the POS Adapter) and local stock operations, updates the Store-Local Inventory, and forwards events through the Outbox; recovery from durable local state must restore inventory access within the 30-minute outage limit.

Store-Local Inventory
The store's own inventory copy, updated locally first so access can continue during communication outages, subject to surviving local services and storage capacity.

Outbox
The durable, ordered queue associated with the Store Agent that retains inventory events until the Communications Front-End acknowledges durable acceptance, supporting the requirement for guaranteed delivery; delivery depends on surviving storage, available capacity and eventual reconnection.

Communications Front-End (CFE)
The publish–subscribe broker at the distribution center that persists each inventory event, acknowledges it, and publishes it by topic to every subscriber; it is the key component of this draft, and its interface is the Inventory Exchange Protocol.

Inventory Exchange Protocol (IEP)
The provided-and-required interface of the Communications Front-End: publish, acknowledge, subscribe and unsubscribe by topic, replay-from-sequence, and resync, over the message types Inventory Event, Acknowledgement, and Subscription.

Inventory Event
One change to a store's or the distribution center's stock — event ID, source, product, quantity delta, timestamp, and per-source sequence number — emitted at up to 100 per second per store.

Acknowledgement
The confirmation the Communications Front-End returns once it has durably persisted an inventory event.

Central Master Inventory
The authoritative record of all store and distribution-center inventory, built by subscribing to every inventory event from every Store Agent and the DC Agent.

DC Agent
The runtime component at the distribution center that publishes the center's own inventory events (goods receipts and shipment dispatches) through the Communications Front-End, so the master record also covers distribution-center stock.

Subscriber
Any component that subscribes to the Communications Front-End; the named subscribers are the Central Master Inventory, Shipping-Order Generation, the Store-Tracking Client, and the Reporting Client, and their number is variable by design.

Shipping-Order Generation
The distribution-center subscriber that computes shipping orders from minimum stock quantities and real-time sales and sends purchase and shipping orders to the supplier system through the Supplier Gateway.

Store-Tracking Client
A subscriber that follows inventory events to track stock at the store level.

Reporting Client
A subscriber that answers manager queries such as lowest stock, overstocked items, and the total value of a product family.

POS Adapter
The boundary element inside the Store Agent that turns the external point-of-sale system's sales records into inventory events.

Supplier Gateway
The boundary element at the distribution center that exchanges orders and goods-receipt confirmations with the external supplier/ERP system.

Store
One supermarket in the regional chain; it holds a local copy of its own inventory and must be able to read it 99% of the time.

Distribution Center (DC)
The central hub that ships product to the stores and hosts the Communications Front-End and the Central Master Inventory.

External system
A system outside our boundary reached through a defined interface — the point-of-sale system at each store and the supplier/ERP system at the distribution center.

Product family
A grouping of products that can be nested to an arbitrary depth.

Scenario ID
The frozen identifier of a quality-attribute scenario (for example, `AV-1`) that the interface specification and the tactics table reference.

Key view
The Components-and-Connectors view of the runtime system delivered in this first draft; the key component it specifies is the Communications Front-End.

Local transaction
A group of changes committed together within one local database, such as an inventory update and its outgoing event, with atomicity, consistency, isolation and durability guarantees.

At-least-once delivery
A delivery model that retries to achieve at least one delivery under the stated failure assumptions and may deliver duplicates; retries retain the original event identifier.

Idempotent processing
Applying an event's business effect once despite repeated delivery, using its stable identifier and atomically recording successful processing with the associated state change.

Per-source sequence
The persistent sequence number assigned within one event source; it establishes that source's ordering and supports gap detection and replay.

Snapshot
A saved representation of state at a known point; recovery may also require subsequent committed history not contained in that snapshot.

Recovery time
The interval from failure occurrence to restored usable service, including detection, restart and state recovery.

Eventual consistency
The property that local and central inventory may temporarily differ but converge when retained events are delivered and processed under the stated recovery assumptions.

Module
An implementation unit of the system's code structure; the element type of the module decomposition view, related to other modules by is-part-of (decomposition) and uses (dependency).

Module decomposition view
The primary view of this draft: the functional decomposition of the system into modules — the Store Subsystem, the Communications Front-End, the Distribution Center Subsystem, and a Common module of shared domain types and the Inventory Exchange Protocol library — from which the runtime components of the key view grow.

6.3 Acronym List

Acronym	Expansion
API	Application programming interface
ATAM	Architecture Tradeoff Analysis Method
C&C	Components and connectors
CFE	Communications Front-End
CI	Continuous integration
DC	Distribution center
ERP	Enterprise resource planning
IEP	Inventory Exchange Protocol
POS	Point of sale
QA	Quality attribute
SAD	Software architecture document
SEI	Software Engineering Institute
TLS	Transport Layer Security
UML	Unified Modeling Language
WAN	Wide-area network